

INDEX RECOMMENDATION

TC-01 to TC-30 Total Status Report

Final result: 30 / 30 PASS

Pass rate: 100%

Suite	Index Recommendation
Endpoint	POST /index-analysis
Report date	20 Aug 2026
Proof source	Textual Swagger JSON response payloads supplied during test execution in this chat

Scope: final test verdicts, key observed proof fields, and corrective retest history.

1. Executive Summary

Overall status: **PASS - 30 of 30 test cases.** The final implementation satisfied the Index Recommendation test suite from TC-01 through TC-30.

Evidence integrity: The proof in this report is derived from the exact response fields pasted into the conversation after Swagger executions. It is textual API-response evidence, not screenshot evidence.

Total	PASS	FAIL (final)	Pass rate	Retest notes
30	30	0	100%	TC-02 and TC-27 required fixes; TC-29 required a corrected test payload

Corrective retest history

- TC-02: Initial behavior recommended a duplicate existing index. The recommendation suppression logic was corrected; retest passed.
- TC-27: Initial threshold gating suppressed candidate_missing_index but still returned recommendation columns, then still returned SQL. Both paths were gated by the threshold; final retest passed.
- TC-29: First execution used an incorrect payload without the existing index and was classified NOT TESTED. The correct payload was executed and passed.

2. Final Status Matrix

ID	Test case	Final proof (key fields)	Status
TC-01	Missing index	candidate_missing_index = true; recommended_index_columns = ["email"]; sequential_scan_detected = true	PASS
TC-02	Index already exists	candidate_missing_index = false; recommended_index_columns = []; index_exists = true	PASS
TC-03	Query uses existing index	index_used = true; index_scan_detected = true; candidate_missing_index = false	PASS
TC-04	Sequential scan on selective filter	selective_filter = true; sequential_scan_detected = true; candidate_missing_index = true	PASS
TC-05	Frequently filtered column	candidate_missing_index = true; recommended_index_columns = ["email"]; filter_index_relevant = false	PASS
TC-06	Frequently joined column	candidate_missing_index = true; recommended_index_columns = ["customer_id"]; join_index_relevant = false	PASS
TC-07	Frequently sorted column	candidate_missing_index = true; recommended_index_columns = ["order_date"];	PASS

Index Recommendation - Total Status Report

		order_by_index_relevant = false	
TC-08	Composite filter	candidate_missing_index = true; recommended_index_columns = ["customer_id", "status"]; index_exists = false	PASS
TC-09	Existing composite index	candidate_missing_index = false; recommended_index_columns = []; index_exists = true	PASS
TC-10	Wrong column order	leading_column_match = false; recommended_index_columns = ["customer_id"]; index_exists = true	PASS
TC-11	Multiple candidate columns	candidate_missing_index = true; recommended_index_columns = ["status", "customer_id", "created_at"]; ranked_recommendation_columns = ["customer_id", "status", "created_at"]	PASS
TC-12	High-frequency slow query	candidate_missing_index = true; recommended_index_columns = ["customer_id"]; recommendation_priority = high	PASS
TC-13	Low-frequency query	candidate_missing_index = true; recommended_index_columns = ["customer_id"]; recommendation_priority = low	PASS
TC-14	Performance regression	index_regression_relevant = true; candidate_missing_index = true; recommended_index_columns = ["customer_id"]	PASS
TC-15	Index removed	index_deleted = true; candidate_missing_index = true; recommended_index_columns = ["customer_id"]	PASS
TC-16	Existing index unused	index_exists = true; index_used = false; candidate_missing_index = false	PASS
TC-17	Duplicate recommendation	candidate_missing_index = true; recommended_index_columns = ["customer_id"]; ranked_recommendation_columns = ["customer_id"]	PASS
TC-18	Multiple queries need same index	candidate_missing_index = true; recommended_index_columns = ["customer_id"]; supporting_query_count = 3	PASS
TC-19	Recommendation explanation	candidate_missing_index = true; recommended_index_columns = ["customer_id"]; recommendation_reason = "Index candidate supported by query filter/join/order-by evidence"	PASS

Evidence basis: user-supplied Swagger response payloads in this chat

Index Recommendation - Total Status Report

TC-20	Recommendation SQL	candidate_missing_index = true; recommended_index_columns = ["email"]; recommendation_sql = "CREATE INDEX idx_users_email ON users (email);"	PASS
TC-21	Table name handling	recommended_index_columns = ["customer_id"]; recommendation_sql = "CREATE INDEX idx_customer_orders_customer_id ON customer_orders (customer_id);"	PASS
TC-22	Column name handling	recommended_index_columns = ["order_date"]; recommendation_sql = "CREATE INDEX idx_orders_order_date ON orders (order_date);"	PASS
TC-23	Composite index recommendation	candidate_missing_index = true; recommended_index_columns = ["customer_id", "status"]; recommendation_sql = "CREATE INDEX idx_orders_customer_id_status ON orders (customer_id, status);"	PASS
TC-24	Partial-index candidate	candidate_missing_index = true; recommended_index_columns = ["customer_id"]; partial_index_candidate = true	PASS
TC-25	No useful index opportunity	candidate_missing_index = false; recommended_index_columns = []; recommendation_evidence = []	PASS
TC-26	Recommendation ranking	ranked_recommendation_columns = ["customer_id", "status", "created_at"]; recommendation_scores.customer_id = 5; recommendation_scores.status = 3	PASS
TC-27	Recommendation threshold	candidate_missing_index = false; recommended_index_columns = []; recommendation_sql = null	PASS
TC-28	Recommendation threshold exceeded	candidate_missing_index = true; recommended_index_columns = ["customer_id"]; recommendation_sql = "CREATE INDEX idx_orders_customer_id ON orders (customer_id);"	PASS
TC-29	Index validation	index_exists = true; candidate_missing_index = false; recommended_index_columns = []	PASS
TC-30	Recommendation report	report_generated = true; candidate_missing_index = true; recommended_index_columns = ["customer_id"]	PASS

3. Detailed Proof by Test Case

Each entry below records the final observed proof used for the PASS verdict. Fields are intentionally limited to the assertions that matter for the corresponding test case.

TC-01 - Missing index

Scenario	Slow query filters on an unindexed column.
Test input	users; no indexes; filter=email; Seq Scan
Observed proof	candidate_missing_index = true recommended_index_columns = ["email"] sequential_scan_detected = true index_exists = false
Final status	PASS
Note	Suitable index candidate was produced.

TC-02 - Index already exists

Scenario	Suitable index already exists; duplicate recommendation must be avoided.
Test input	existing email index; index_columns=[email]; filter=email
Observed proof	candidate_missing_index = false recommended_index_columns = [] index_exists = true
Final status	PASS
Note	Initial run exposed duplicate recommendation behavior; minimum fix applied and retest passed.

TC-03 - Query uses existing index

Scenario	Execution plan already uses a suitable existing index.
Test input	idx_users_email used; Index Scan; filter=email
Observed proof	index_used = true index_scan_detected = true candidate_missing_index = false recommended_index_columns = []
Final status	PASS
Note	No unnecessary recommendation generated.

TC-04 - Sequential scan on selective filter

Scenario	Selective query performs a sequential scan.
Test input	table_row_count=100000; result_row_count=100; filter=customer_id; Seq Scan
Observed proof	selective_filter = true sequential_scan_detected = true candidate_missing_index = true recommended_index_columns = ["customer_id"]
Final status	PASS
Note	Selective filter evidence supported a missing-index candidate.

TC-05 - Frequently filtered column

Scenario	Column is repeatedly used in WHERE/filter conditions.
Test input	no index; filter=email; Seq Scan
Observed proof	candidate_missing_index = true recommended_index_columns = ["email"] filter_index_relevant = false sequential_scan_detected = true
Final status	PASS
Note	Filter column was considered for indexing.

TC-06 - Frequently joined column

Scenario	Column is repeatedly used as a JOIN key.
Test input	no index; join=customer_id; Seq Scan
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id"] join_index_relevant = false sequential_scan_detected = true
Final status	PASS
Note	Join column was considered for indexing.

TC-07 - Frequently sorted column

Scenario	Column is repeatedly used for ORDER BY.
Test input	no index; order_by=order_date; Seq Scan
Observed proof	candidate_missing_index = true recommended_index_columns = ["order_date"] order_by_index_relevant = false sequential_scan_detected = true
Final status	PASS
Note	Sort column was considered for indexing.

TC-08 - Composite filter

Scenario	Query filters on multiple columns.
Test input	no indexes; filters=[customer_id,status]; Seq Scan
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id", "status"] index_exists = false
Final status	PASS
Note	Composite candidate columns were generated.

TC-09 - Existing composite index

Scenario	Suitable composite index already exists.
Test input	idx_orders_customer_status; index_columns=[customer_id,status]; same filters
Observed proof	candidate_missing_index = false recommended_index_columns = [] index_exists = true index_type = composite
Final status	PASS
Note	Duplicate composite recommendation was suppressed.

TC-10 - Wrong column order

Scenario	Existing composite index has an unsuitable leading column.
Test input	existing_index_columns=[status,customer_id]; query/filter starts customer_id
Observed proof	leading_column_match = false recommended_index_columns = ["customer_id"] index_exists = true index_type = composite
Final status	PASS
Note	Alternative candidate was produced because leading-column order was unsuitable.

TC-11 - Multiple candidate columns

Scenario	Several columns compete as index candidates.
Test input	filters=[status,customer_id]; join=[customer_id]; order_by=[created_at]
Observed proof	candidate_missing_index = true recommended_index_columns = ["status", "customer_id", "created_at"] ranked_recommendation_columns = ["customer_id", "status", "created_at"]
Final status	PASS
Note	Ranking favored the column supported by both filter and join evidence.

TC-12 - High-frequency slow query

Scenario	A slow query executes frequently and should receive higher priority.
Test input	filter=customer_id; Seq Scan; query_execution_count=50; query_slow=true
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id"] recommendation_priority = high
Final status	PASS
Note	High priority assigned as designed.

TC-13 - Low-frequency query

Scenario	A slow query executes rarely and should receive lower priority.
Test input	filter=customer_id; Seq Scan; query_execution_count=2; query_slow=true
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id"] recommendation_priority = low
Final status	PASS
Note	Low priority assigned for low execution frequency.

TC-14 - Performance regression

Scenario	Query performance regressed after an index/plan change.
Test input	index_deleted=true; regression_detected=true; count=20; slow=true; Seq Scan
Observed proof	index_regression_relevant = true candidate_missing_index = true recommended_index_columns = ["customer_id"] recommendation_priority = high sequential_scan_detected = true
Final status	PASS
Note	Regression and index-removal evidence were surfaced together.

TC-15 - Index removed

Scenario	Removing an index causes a sequential scan and a replacement candidate is needed.
Test input	index_deleted=true; no indexes; filter=customer_id; Seq Scan
Observed proof	index_deleted = true candidate_missing_index = true recommended_index_columns = ["customer_id"] sequential_scan_detected = true index_exists = false
Final status	PASS
Note	Equivalent index candidate was recommended.

TC-16 - Existing index unused

Scenario	An index exists but is not selected by the plan.
Test input	idx_orders_customer_id exists; used_indexes=[]; filter=customer_id; Seq Scan
Observed proof	index_exists = true index_used = false candidate_missing_index = false recommended_index_columns = [] index_usage_investigation_required = true index_not_chosen = true
Final status	PASS
Note	System avoided blindly recommending a duplicate index.

TC-17 - Duplicate recommendation

Scenario	Same candidate appears repeatedly across evidence.
Test input	filter=[customer_id,customer_id]; join=[customer_id]
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id"] ranked_recommendation_columns = ["customer_id"]
Final status	PASS
Note	Candidate was deduplicated.

TC-18 - Multiple queries need same index

Scenario	Several query instances support the same index candidate.
Test input	3 supporting queries; all filter customer_id
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id"] supporting_query_count = 3
Final status	PASS
Note	Common recommendation was backed by three supporting queries.

TC-19 - Recommendation explanation

Scenario	Generated recommendation must include reason/evidence.
Test input	no indexes; filter=customer_id; Seq Scan
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id"] recommendation_reason = "Index candidate supported by query filter/join/order-by evidence" recommendation_evidence = ["customer_id"]
Final status	PASS
Note	Human-readable reason and evidence were returned.

TC-20 - Recommendation SQL

Scenario	Generated candidate must include executable CREATE INDEX SQL.
Test input	users; no indexes; filter=email; Seq Scan
Observed proof	candidate_missing_index = true recommended_index_columns = ["email"] recommendation_sql = "CREATE INDEX idx_users_email ON users (email);"
Final status	PASS
Note	SQL statement matched table and column.

TC-21 - Table name handling

Scenario	Recommendation SQL must preserve a normal table name correctly.
Test input	table=customer_orders; filter=customer_id
Observed proof	recommended_index_columns = ["customer_id"] recommendation_sql = "CREATE INDEX idx_customer_orders_customer_id ON customer_orders (customer_id);"
Final status	PASS
Note	Table name was handled correctly.

TC-22 - Column name handling

Scenario	Recommendation SQL must preserve a normal column name correctly.
Test input	table=orders; filter=order_date
Observed proof	recommended_index_columns = ["order_date"] recommendation_sql = "CREATE INDEX idx_orders_order_date ON orders (order_date);"
Final status	PASS
Note	Column name was handled correctly.

TC-23 - Composite index recommendation

Scenario	Multiple predicates justify a composite index candidate.
Test input	table=orders; filters=[customer_id,status]; Seq Scan
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id", "status"] recommendation_sql = "CREATE INDEX idx_orders_customer_id_status ON orders (customer_id, status);"
Final status	PASS
Note	Composite CREATE INDEX statement was generated.

TC-24 - Partial-index candidate

Scenario	Stable predicate with repeated execution can justify a partial-index candidate.
Test input	count=20; stable_predicate="status = 'ACTIVE'"; filter=customer_id
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id"] partial_index_candidate = true partial_index_predicate = "status = 'ACTIVE'"
Final status	PASS
Note	Stable predicate was surfaced as partial-index evidence.

TC-25 - No useful index opportunity

Scenario	Query has no useful filter/join/order-by indexing opportunity.
Test input	no filter/join/order_by; Seq Scan
Observed proof	candidate_missing_index = false recommended_index_columns = [] recommendation_evidence = [] recommendation_sql = null partial_index_candidate = false
Final status	PASS
Note	No false recommendation was generated.

TC-26 - Recommendation ranking

Scenario	Multiple candidates must be ranked by defined scoring.
Test input	filter=[status,customer_id]; join=[customer_id]; order_by=[created_at]
Observed proof	ranked_recommendation_columns = ["customer_id", "status", "created_at"] recommendation_scores.customer_id = 5 recommendation_scores.status = 3 recommendation_scores.created_at = 1
Final status	PASS
Note	Scoring used filter=3, join=2, order-by=1.

TC-27 - Recommendation threshold

Scenario	Candidate below configured threshold must not become a recommendation.
Test input	recommendation_score=2; recommendation_threshold=5; filter=customer_id
Observed proof	candidate_missing_index = false recommended_index_columns = [] recommendation_sql = null
Final status	PASS
Note	Initial threshold test exposed two gating issues; recommendation list and SQL were fixed, then retest passed.

TC-28 - Recommendation threshold exceeded

Scenario	Candidate meeting/exceeding threshold must generate a recommendation.
Test input	recommendation_score=7; recommendation_threshold=5; filter=customer_id
Observed proof	candidate_missing_index = true recommended_index_columns = ["customer_id"] recommendation_sql = "CREATE INDEX idx_orders_customer_id ON orders (customer_id);"
Final status	PASS
Note	Final retest evidence was captured in chat.

TC-29 - Index validation

Scenario	Recommendation must be validated against current schema and duplicate candidates rejected.
Test input	existing_idx_orders_customer_id; index_columns=[customer_id]; score=7 threshold=5
Observed proof	index_exists = true candidate_missing_index = false recommended_index_columns = [] recommendation_sql = null
Final status	PASS
Note	First attempt used the wrong payload and was not counted; correct retest passed.

TC-30 - Recommendation report

Scenario	Completed analysis must return a clear recommendation report.
Test input	filter=customer_id; count=20; slow=true; score=7 threshold=5
Observed proof	report_generated = true candidate_missing_index = true recommended_index_columns = ["customer_id"] recommendation_priority = high recommendation_reason = "Index candidate supported by query filter/join/order-by evidence" recommendation_sql = "CREATE INDEX idx_orders_customer_id ON orders (customer_id);"
Final status	PASS
Note	Final report contains status, priority, explanation, and SQL.

4. Final Conclusion

FINAL STATUS: 30 / 30 PASS (100%)

The final Index Recommendation implementation passed all planned functional cases after the documented corrective retests. The evidence set now includes the TC-28 threshold-exceeded retest and the corrected TC-29 schema-validation execution.

Validated capabilities include: missing-index detection, duplicate suppression, plan-awareness, selective-filter analysis, filter/join/order-by candidates, composite recommendations, candidate ranking, priority, explanations, CREATE INDEX SQL, partial-index evidence, recommendation thresholds, schema validation, and final report generation.

Audit note: This document summarizes response evidence supplied during the interactive test run. It does not assert independent database execution outside those supplied API responses.

INDEX RECOMMENDATION

Edge Case Test Status Report

EC-01 to EC-35

Final Result	35 / 35 PASS
Pass Rate	100%
Database terminology	SQLite
Evidence basis	Swagger response payloads supplied during interactive testing

Final status reflects the latest successful retest for cases that initially failed.

1. Executive Summary

Overall status: 35/35 **PASS**. All EC-01 through EC-35 scenarios were executed and the final observed outputs satisfied the defined edge-case expectations.

Retest history: EC-08, EC-15, EC-18, EC-19, EC-22, EC-25, EC-27, EC-29, EC-31, EC-32 required one or more minimum fixes before the final PASS result. The proof shown in this report is the final successful retest evidence.

Terminology note: EC-22 is reported as 'SQLite unavailable'. PostgreSQL terminology is intentionally not used.

2. Final Status Matrix

ID	Scenario	Final Status	Key Proof
EC-01	Index already exists	PASS	index_exists: true; candidate_missing_index: false
EC-02	Table has no indexes	PASS	index_exists: false; candidate_missing_index: true
EC-03	Too many indexes	PASS	multiple_indexes: true; index_exists: true
EC-04	Duplicate indexes	PASS	has_duplicate_indexes: true; duplicate_indexes: ["idx_orders_customer_id_copy"]
EC-05	Overlapping indexes	PASS	has_similar_indexes: true; similar_indexes: ["idx_orders_customer_status"]
EC-06	Composite index exists	PASS	index_type: "composite"; leading_column_match: true
EC-07	Wrong column order	PASS	index_type: "composite"; leading_column_match: false
EC-08	Query returns most rows	PASS	selective_filter: false; candidate_missing_index: false
EC-09	Query returns very few rows	PASS	selective_filter: true; candidate_missing_index: true
EC-10	Query rarely executed	PASS	recommendation_priority: "low"; candidate_missing_index: true
EC-11	Query executed very frequently	PASS	recommendation_priority: "high"; candidate_missing_index: true
EC-12	Existing index unused	PASS	index_usage_investigation_required: true; index_not_chosen: true
EC-13	Recently created index	PASS	insufficient_observation_period: true; unused_indexes: []
EC-14	Missing statistics	PASS	usage_level: "unknown"; unused_indexes: []
EC-15	Multiple candidate indexes	PASS	candidate_missing_index: true; ranked_recommendation_columns: ["customer_id", "status", "created_at"]
EC-16	Same candidate from multiple queries	PASS	recommended_index_columns: ["customer_id"]; supporting_query_count: 3
EC-17	Same query generates repeated recommendation	PASS	recommended_index_columns: ["customer_id"]; ranked_recommendation_columns: ["customer_id"]
EC-18	Table does not exist	PASS	table_status: "missing"; candidate_missing_index: false
EC-19	Column does not exist	PASS	candidate_missing_index: false; recommended_index_columns: []
EC-20	Column renamed	PASS	previous_index_columns: ["customer_id"]; index_columns: ["customer_ref"]
EC-21	Permission denied	PASS	metadata_permission_denied: true; permission_status: "permission denied"
EC-22	SQLite unavailable	PASS	metadata_error: true; metadata_status: "collection error"
EC-23	Very large table	PASS	large_table: true; selective_filter:

			true
EC-24	Very wide table	PASS	wide_table: true; analysis_scope: "reduced"
EC-25	Partial index candidate	PASS	partial_index_candidate: true; partial_index_predicate: "status = 'ACTIVE'"
EC-26	Expression index candidate	PASS	expression_index_candidate: true; expression_index_expression: "LOWER(email)"
EC-27	Unique constraint already covers access	PASS	candidate_missing_index: false; recommended_index_columns: []
EC-28	Foreign-key column	PASS	join_index_relevant: true; index_exists: true
EC-29	Query has functions on column	PASS	expression_index_candidate: false; expression_index_expression: ""
EC-30	Parameterized query	PASS	query_template: "SELECT * FROM orders WHERE customer_id = ?"; query_fingerprint: "orders_customer_id_eq_param"
EC-31	SQL injection-like identifier input	PASS	candidate_missing_index: false; recommended_index_columns: []
EC-32	Stale recommendation	PASS	recommendation_revalidation_required: true; candidate_missing_index: false
EC-33	Recommendation would create excessive write overhead	PASS	write_overhead_tradeoff: true; candidate_missing_index: true
EC-34	Index storage cost is high	PASS	index_size_mb: 2048; large_index: true
EC-35	Recommendation improves one query but harms workload	PASS	workload_impact_tradeoff: true; candidate_missing_index: true

3. Detailed Proof Evidence

Each case below lists the final observed proof fields used to assign PASS. **These are final-state observations, not inferred results.**

EC-01 - Index already exists **PASS**

```
index_exists: true
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null
```

Result: Duplicate index recommendation was suppressed.

EC-02 - Table has no indexes **PASS**

```
index_exists: false
candidate_missing_index: true
recommended_index_columns: ["customer_id"]
recommendation_sql: "CREATE INDEX idx_orders_customer_id ON orders (customer_id);"
```

Result: A supported missing-index candidate was generated.

EC-03 - Too many indexes **PASS**

```
multiple_indexes: true
index_exists: true
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null
```

Result: The analyzer did not blindly add another index.

EC-04 - Duplicate indexes **PASS**

```
has_duplicate_indexes: true
duplicate_indexes: ["idx_orders_customer_id_copy"]
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null
```

Result: Existing duplication was surfaced and no further duplicate recommendation was created.

EC-05 - Overlapping indexes **PASS**

```
has_similar_indexes: true
similar_indexes: ["idx_orders_customer_status"]
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null
```

Result: Overlapping coverage was considered before recommending a new index.

EC-06 - Composite index exists **PASS**

```
index_type: "composite"
leading_column_match: true
index_used: true
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null
```

Result: Composite index order and actual use were recognized.

EC-07 - Wrong column order **PASS**

```
index_type: "composite"
leading_column_match: false
recommended_index_columns: ["customer_id"]
recommendation_sql: null
sequential_scan_detected: true
```

Result: A composite index with the wrong leading column was not treated as full coverage.

EC-08 - Query returns most rows **PASS**

```
selective_filter: false
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null
```

Result: Final retest passed after selectivity gating was applied to candidate, columns, and SQL generation. *[Final retest after minimum fix]*

EC-09 - Query returns very few rows **PASS**

```
selective_filter: true
candidate_missing_index: true
recommended_index_columns: ["customer_id"]
recommendation_sql: "CREATE INDEX idx_orders_customer_id ON orders (customer_id);"
```

Result: Highly selective access justified a candidate.

EC-10 - Query rarely executed **PASS**

```
recommendation_priority: "low"
candidate_missing_index: true
recommended_index_columns: ["customer_id"]
```

Result: Rare execution lowered the recommendation priority.

EC-11 - Query executed very frequently **PASS**

```
recommendation_priority: "high"
candidate_missing_index: true
recommended_index_columns: ["customer_id"]
```

Result: Frequent slow execution raised the recommendation priority.

EC-12 - Existing index unused **PASS**

```
index_usage_investigation_required: true
index_not_chosen: true
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null
```

Result: The endpoint requested investigation instead of producing a duplicate index.

EC-13 - Recently created index **PASS**

```
insufficient_observation_period: true
unused_indexes: []
index_not_chosen: false
index_exists: true
```

Result: Insufficient history prevented a premature unused-index conclusion.

EC-14 - Missing statistics **PASS**

```
usage_level: "unknown"  
unused_indexes: []  
index_not_chosen: false
```

Result: Missing usage statistics resulted in an unknown status rather than an unsupported conclusion.

EC-15 - Multiple candidate indexes **PASS**

```
candidate_missing_index: true  
ranked_recommendation_columns: ["customer_id", "status", "created_at"]  
recommendation_scores: {"customer_id": 5, "status": 3, "created_at": 1}
```

Result: Final retest passed after the ranked recommendation field was restored. *[Final retest after minimum fix]*

EC-16 - Same candidate from multiple queries **PASS**

```
recommended_index_columns: ["customer_id"]  
supporting_query_count: 3  
recommendation_evidence: ["customer_id"]
```

Result: Evidence from multiple queries was consolidated around one candidate.

EC-17 - Same query generates repeated recommendation **PASS**

```
recommended_index_columns: ["customer_id"]  
ranked_recommendation_columns: ["customer_id"]  
recommendation_evidence: ["customer_id"]  
recommendation_sql: "CREATE INDEX idx_orders_customer_id ON orders (customer_id);"
```

Result: Repeated occurrences were deduplicated.

EC-18 - Table does not exist **PASS**

```
table_status: "missing"  
candidate_missing_index: false  
recommended_index_columns: []  
recommendation_sql: null
```

Result: Final retest passed after recommendations and SQL were blocked for missing tables. *[Final retest after minimum fix]*

EC-19 - Column does not exist **PASS**

```
candidate_missing_index: false  
recommended_index_columns: []  
ranked_recommendation_columns: []  
recommendation_sql: null
```

Result: Final retest rejected the invalid column as a recommendation candidate. *[Final retest after minimum fix]*

EC-20 - Column renamed **PASS**

```
previous_index_columns: ["customer_id"]  
index_columns: ["customer_ref"]  
metadata_refreshed: true
```

Result: Schema metadata refresh detected the renamed column.

EC-21 - Permission denied **PASS**

```
metadata_permission_denied: true
```

permission_status: "permission denied"

Result: Permission failure was returned clearly without a crash.

EC-22 - SQLite unavailable **PASS**

metadata_error: true
metadata_status: "collection error"
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null

Result: Final retest passed after SQLite metadata-collection errors were made to suppress recommendations.

[Final retest after minimum fix]

EC-23 - Very large table **PASS**

large_table: true
selective_filter: true
report_generated: true
candidate_missing_index: true
recommended_index_columns: ["customer_id"]

Result: Analysis completed stably on a very large table with selective access.

EC-24 - Very wide table **PASS**

wide_table: true
analysis_scope: "reduced"
report_generated: true

Result: The endpoint switched to reduced analysis scope for a wide table.

EC-25 - Partial index candidate **PASS**

partial_index_candidate: true
partial_index_predicate: "status = 'ACTIVE'"
candidate_missing_index: true

Result: Final retest exposed an explicit partial-index candidate flag and predicate. *[Final retest after minimum fix]*

EC-26 - Expression index candidate **PASS**

expression_index_candidate: true
expression_index_expression: "LOWER(email)"

Result: A supported expression index candidate was represented explicitly.

EC-27 - Unique constraint already covers access **PASS**

candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null

Result: Final retest suppressed redundant index creation when a unique constraint already covered access.

[Final retest after minimum fix]

EC-28 - Foreign-key column **PASS**

join_index_relevant: true
index_exists: true
candidate_missing_index: false
recommended_index_columns: []

recommendation_sql: null

Result: Existing join/foreign-key coverage was considered before new recommendations.

EC-29 - Query has functions on column **PASS**

expression_index_candidate: false
expression_index_expression: ""
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null

Result: Final retest prevented a normal index recommendation when the expression form was unsupported.

[Final retest after minimum fix]

EC-30 - Parameterized query **PASS**

query_template: "SELECT * FROM orders WHERE customer_id = ?"
query_fingerprint: "orders_customer_id_eq_param"
query_identity: "orders_customer_id_eq_param"

Result: The query was identified by a stable template/fingerprint rather than raw parameter values.

EC-31 - SQL injection-like identifier input **PASS**

candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null

Result: Final retest blocked executable recommendation SQL for unsafe identifier input. *[Final retest after minimum fix]*

EC-32 - Stale recommendation **PASS**

recommendation_revalidation_required: true
candidate_missing_index: false
recommended_index_columns: []
recommendation_sql: null

Result: Final retest required revalidation and suppressed stale recommendation output. *[Final retest after minimum fix]*

EC-33 - Recommendation would create excessive write overhead **PASS**

write_overhead_tradeoff: true
candidate_missing_index: true

Result: The write-overhead trade-off was explicitly surfaced.

EC-34 - Index storage cost is high **PASS**

index_size_mb: 2048
large_index: true
index_exists: true
recommended_index_columns: []
recommendation_sql: null

Result: High index storage cost was identified and no redundant recommendation was produced.

EC-35 - Recommendation improves one query but harms workload **PASS**

workload_impact_tradeoff: true
candidate_missing_index: true
recommended_index_columns: ["customer_id"]

Result: Workload-wide harm was explicitly flagged even though the single-query candidate remained visible.

4. Corrective Retest Ledger

The following cases had an initial mismatch and were not marked PASS until the corrected behavior was observed:

EC-08 - Query returns most rows: Final retest passed after selectivity gating was applied to candidate, columns, and SQL generation.

EC-15 - Multiple candidate indexes: Final retest passed after the ranked recommendation field was restored.

EC-18 - Table does not exist: Final retest passed after recommendations and SQL were blocked for missing tables.

EC-19 - Column does not exist: Final retest rejected the invalid column as a recommendation candidate.

EC-22 - SQLite unavailable: Final retest passed after SQLite metadata-collection errors were made to suppress recommendations.

EC-25 - Partial index candidate: Final retest exposed an explicit partial-index candidate flag and predicate.

EC-27 - Unique constraint already covers access: Final retest suppressed redundant index creation when a unique constraint already covered access.

EC-29 - Query has functions on column: Final retest prevented a normal index recommendation when the expression form was unsupported.

EC-31 - SQL injection-like identifier input: Final retest blocked executable recommendation SQL for unsafe identifier input.

EC-32 - Stale recommendation: Final retest required revalidation and suppressed stale recommendation output.

5. Final Conclusion

FINAL STATUS: 35 / 35 PASS (100%)

Coverage includes: duplicate/overlapping indexes, selectivity, execution frequency, usage history, missing statistics, schema changes, SQLite metadata failures, large/wide tables, partial/expression indexes, constraints, parameterized queries, unsafe identifiers, stale recommendations, and workload/storage trade-offs.

Evidence note: Proof values were taken from the Swagger response payloads supplied during the EC-01 through EC-35 test sequence in this conversation.